

Leveraging Machine Learning Techniques for Architecting Self-Adaptive IoT Systems

Henry Muccini
University of L'Aquila
L'Aquila, Italy
henry.muccini@univaq.it

Karthik Vaidhyanathan
Gran Sasso Science Institute
L'Aquila, Italy
karthik.vaidhyanathan@gssi.it

Abstract—The use of IoT systems is increasing day by day. However, these systems due to their heterogeneity and inherently dynamic nature, face different uncertainties from the context, environment, etc. Such uncertainties can have a big impact on the overall system QoS, especially on energy efficiency and data traffic. This calls for better ways of architecting IoT systems that may self-adapt to keep the desired QoS. This paper presents an approach that leverages the use of machine learning (ML) techniques to perform a proactive adaptation of IoT architectures using self-adaptation patterns. It i) continuously monitors the QoS parameters; ii) forecasts possible deviations from the acceptable QoS parameters; iii) selects the best adaptation pattern based on forecasts using reinforcement learning (RL) techniques; iv) checks the quality of the selected decision using feedback mechanisms; and v) continuously performs the loop of the forecast, adaptation, and feedback. The results of our evaluations show that our approach can provide accurate QoS forecasts and further improve the energy efficiency of the system while maintaining the required data traffic.

Index Terms—Self-adaptation, Machine learning, IoT architectures, Proactive decision making, Reinforcement learning

I. INTRODUCTION

As per Cisco, around 30 billion IoT devices will be connected by the end of 2020 [1]. However, a 2017 IoT survey from Cisco [2] states that about 75% of self-initiated IoT projects from the companies were considered as a *failure* and one of the reason was attributed to the lack of proper software engineering practices. IoT systems, being inherently dynamic and heterogeneous in nature, face different uncertainties from context, environment, etc. This, in turn, can have major implications on the system QoS [3]. In fact, ensuring QoS is considered as one of the important challenges in architecting IoT systems [4] [5].

Self-adaptation mechanisms have been emerging as a promising solution towards managing run-time uncertainties so to improve the system QoS [6], [7]. However, one of the issues with the existing solutions is that most of them are *reactive*, where adaptation is carried out in the event of uncertainties [8]. Moreover, current adaptation methods: i) do not provide the systems the ability to proactively identify the need for adaptation with good accuracy [8]; ii) may allow to temporally overcome an upcoming failure, while not preventing the system from entering the same state in future. This is better known as self-adaptation through *Achieving* and it has not been explored much in literature [6].

Considering the increasing number of IoT devices, there will be a tremendous increase in the amount of data traffic generated as well as energy consumed by these systems. Therefore, reducing energy consumption and data traffic are considered as two of the most important QoS challenges in IoT [4] [5] [9]. Energy efficiency has even become a global concern with ICT expected to consume around 21% of the world's electricity by 2030 [12]. This holds for even data traffic as the global carbon footprint is expected to increase with an exponential increase in ICT data traffic [13].

Some methods to reduce energy consumption consists of reducing the sensor acquisition frequency, using decentralized architecture, etc. However, such solutions may affect service accuracy (reduced frequency) or an increase in traffic (decentralized architecture). This mandates the use of structural architectural adaptation techniques that, *while offering the desired energy efficiency, maintain data traffic between the required performance thresholds*. For this purpose, we rely on three different self-adaptation patterns for architecting IoT systems proposed in [11]. Since the usage of every pattern has a different impact on QoS depending on the operational context of the system, we make use of RL techniques to find the best adaptation strategy to ensure that the architecture learns with every adaptation performed.

Hence, the novel contribution of this paper is an approach to architect self-adaptive IoT systems that uses a combination of ML and RL techniques for performing proactive adaptations and thereby improve the energy efficiency of the system without compromising the overall performance. Given a scenario with a set of energy and data traffic constraints, our approach i) uses ML models, specifically Long Short Term Memory (LSTM) networks, to forecast the energy consumption and data traffic; ii) performs adaptation based on the forecasts by selecting the best adaptation pattern using an RL based algorithm; iii) uses feedback mechanisms to continuously improve the process of adaptations thereby making it more smarter and efficient.

The evaluation results of our approach on the NdR case study (<https://nottedeiricercatoriaq.it/>) show that our approach can produce accurate energy and data traffic forecasts and further execute effective and efficient adaptations which significantly improves the energy efficiency while maintaining the required data traffic.

II. BACKGROUND, CASE STUDY AND MOTIVATION

A. Self-Adaptation Patterns in IoT Systems

Three different patterns for self-adaptation in cyber-physical systems were identified by Musil et al. [11]. These include :

1. Synthesize-Utilize (SU): This is a completely decentralized pattern which provides each of the sensor nodes, the ability to perform decision making without resorting to an external controller.

2. Synthesize-Command (SC): This is a centralized pattern in which all the sensor nodes and other components communicate to a central coordinator, which will then perform the decision-making process.

3. Collect-Organize (CO): This is a semi-decentralized pattern that uses additional controller components to receive the data from the sensor nodes and for decision making.

B. Scientific Exhibition: A Case Study

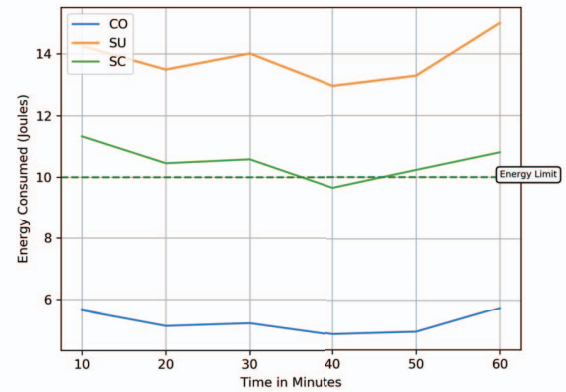
The NdR street fair is a scientific exhibition event organized by the University of L'Aquila. In this event, the research community and public are brought together (around 25,000 participants) from the afternoon until late at night to share a combination of entertainment and information. This event takes place throughout the entire city. As an example of a demonstrative scenario, we will consider the NdR activities held in the city center, in which performances, lectures, demonstrations, workshops take place in its squares, main streets, and buildings.

Our research group has been invited to develop an IoT-based solution to improve the quality of the visiting experience. Without the loss of generality, in this paper, we focus on two such IoT services planned for NdR. These services are related to the automated venue and parking lot management of 3 venues (capacity 500, 200, 200) and 2 parking lots (capacity 100, 75). One important thing to note is that these parking lots and venues are created in an ad-hoc way since it is a one-day event, and hence the availability of external power sources and legacy infrastructures is extremely limited.

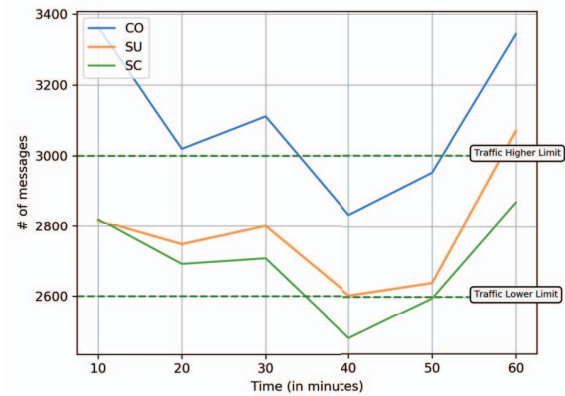
The system uses sensors such as people counter, RFID reader, parking mats at the venue, and parking entrances/exits to provide real-time information on the venue and parking lots availability. These data are then ingested into a central database which is further processed to redirect the users to different parking lots based on their venue preferences. Each of these sensors operates in two different operational modes: i) *normal mode* in which the sensor components gather data at standard frequency rate (normal operational scenario); ii) *critical mode* during which the sensor components gather data at a higher frequency rate (when there is a sudden increase in the crowd/vehicle arrival/departure rate, the data needs to be gathered at a faster rate to ensure service accuracy).

C. Motivation

One of the important constraints we have in the NdR case study is to ensure that the sensors can operate in an energy-efficient manner. This is because they are battery powered and can run only for half a day with a normal battery setup.



(a) Energy Consumption of Sensors



(b) Data Traffic in the System

Fig. 1: Simulation results of NdR architecture

Even though charging can be performed, it might happen that during peak hours the charge goes down, and plugging in a new sensor (or charging the existing one) can be costly. Moreover, we also need to ensure that the overall data traffic of the system is optimized since high data traffic can affect performance while little data traffic might cause issues with service accuracy. This is illustrated in Figure 1. We can clearly observe that patterns that are good for energy efficiency are not performing well when it comes to data traffic.

Henceforth, an ideal IoT system should have the ability to proactively predict the consumption and intelligently use different self-adaptation patterns based on the context to keep the energy and data traffic consumption within the limits. Moreover, it may happen that a particular pattern selected behaves indifferently (due to sudden mode changes/environmental uncertainties). Thus, the system should have the ability to learn from these situations and continuously improve the decision-making process to ensure better efficiency.

III. METHODOLOGY

Figure 2 shows an overview of the approach. It is built on top of the MAPE-K adaptation loop [14].

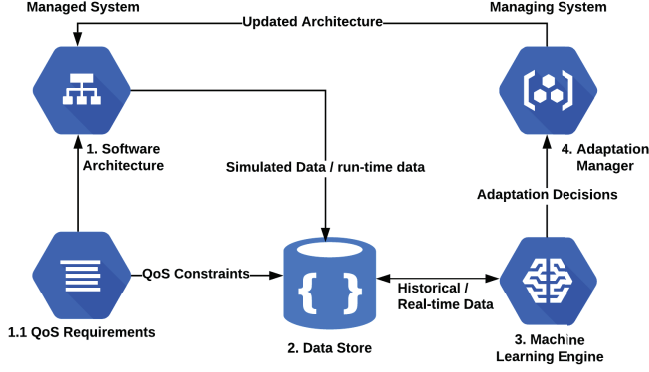


Fig. 2: Approach Overview

Software Architecture : In our approach, we model the architecture such that it will be able to switch between multiple patterns in run-time. In this phase, the operational modes of sensor components along with the conditions for transitions between the modes are defined.

QoS Requirements: QoS requirements for energy and data traffic in the form of thresholds are defined by the respective stakeholders through a configuration file. The process involves defining four variables for a given time-period: i) high energy (*he*), the maximum energy that can be consumed by the sensors; ii) low energy (*le*), the threshold below which the sensors save the maximum energy; iii) high data traffic (*hd*), the maximum allowed traffic to avoid congestion; iv) low data traffic (*ld*), the minimum traffic required to maintain service accuracy. These are then stored in the *Data store*.

Data Store: It consists of run-time execution logs. These logs contain information on the energy consumed and the number of messages exchanged by the components of the architecture at every instant of time. This is then further processed. This represents the *Monitor* phase of MAPE-K.

Machine Learning Engine (MLE): It is responsible for identifying the need for adaptation at an early stage and for selecting the best adaptation pattern to be used. Figure 3 shows the data pipeline of the ML engine. It explodes the third step presented in Figure 2. It consists of two phases, *Build Phase* and *Operational Phase*. The former refers to the process of building ML models for forecasting QoS parameters. The latter refers to the run-time phase where the ML models are used to forecast the QoS metrics based on the run-time data and select the best adaptation pattern for architectural adaptation. When running the *Operational Phase*, the *Build Phase* is run in parallel (at regular intervals) to ensure that the ML model is up-to-date and thus avoid problems of concept drift. It further consists of different components:

Data Store: Data store (Component 1) contains the historical data (execution logs) for the build phase (the data store represented in Figure 3 is the one represented in Figure 2). These logs contain data for different intervals of time and may not be uniform.

Feature Extractor: It (component 2) first splits the data in

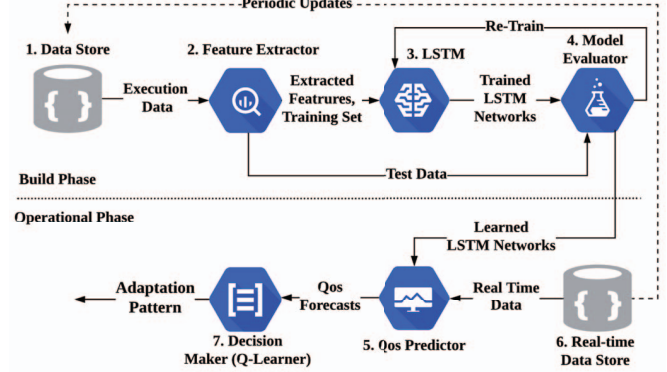


Fig. 3: Machine Learning Engine Pipeline

the *Data store* into two datasets, namely energy data (multivariate - the energy of each component) and traffic data (univariate - overall traffic). It then applies four pre-processing steps on each dataset namely, i) Time series Modeling, ii) Normalization, iii) Conversion to Supervised learning problem, and the iv) Train and Test data generation [15]. These processes create time-series datasets and further splits them in the ratio 7:3 where 70% forms the training set and 30% forms the testing set. The training set is sent to the LSTM component and testing set to the Model Evaluator.

LSTM and Model Evaluator: The LSTM (component 3) forms one of the key components of the approach as it is responsible for building the forecasting models. It creates two forecasting models for i) Forecasting the total energy consumption of the sensor components; ii) Forecasting the data traffic of the system. LSTM Networks belongs to the class of Recurrent Neural Networks (RNN) [16]. They have shown to be very effective in time-series forecasting [18] as they can handle the problem of long-term dependency better known in the literature as the *Vanishing Gradient Problem* [17], as compared to traditional RNN's. In our previous work [15], we have shown how LSTM networks can be used for forecasting energy consumption and why they are more effective when compared to traditional models like ARIMA. In this work, we use the same approach to create the LSTM models but for forecasting both energy consumption and data traffic. The models are further evaluated for accuracy by the *Model Evaluator* (component 4). It performs this by using the test data sets. In the case of lower accuracy, retraining is performed by modifying the network parameters and a trained network forms the *QoS Predictor*.

Real Time Data Store and QoS Predictor: During the operational phase, the real-time execution data is ingested into the real-time data store (component 6) and this is immediately sent to the *QoS predictor*, which keeps aggregating the data until the number of observations, n becomes equal to the lag, l (the number of observations required for generating the forecast). It then uses the trained LSTM networks to forecast the energy and data traffic values for the next H time steps.

Decision Maker (Q-Learning): The decision-maker com-

ponent (component 7) is responsible for selecting the best adaptation pattern based on the forecasts provided by the *QoS predictor*. It uses Q-Learning, an RL based technique [19] to perform the decision making.

The problem of selecting the best adaptation pattern can be converted to a famous "Robot in the Grid World" problem [20], where the software architecture can be considered as a robot that has to navigate through a grid, and the goal is to reach to a position in the grid which enables the system to become efficient with respect to energy and data traffic thresholds as defined in the *QoS requirements* (Figure 2). The first part of Q-Learning is to divide the problem into a set of *states and actions*. The forecasts of the *QoS predictor* can be classified into different categories based on the QoS thresholds. The state-space then represents a set that contains all the categories based on the thresholds combined with each pattern, p available. For instance, if there are three patterns $\{p_1, p_2, p_3\}$, and QoS thresholds = $\{he, hd, le, ld\}$ then state set can be: $S = \{he_hd_p1, he_hd_p2, \dots, le_ld_p2\}$. The action space represents the switch to different patterns. Let $P = \{p_1, p_2, \dots, p_m\}$ represent the set of available patterns then, action space, A can be defined as $A = \{0, 1, \dots, m\}$ where any $i \in A$ represents switching to the pattern $p_i \in P$.

S and A together forms an $N \times M$ *Q-Table* where N represents the states and M , the actions. Each value inside *Q-Table* corresponds to a (state s , action a) pair and it indicates the relevance of taking an action a from state s . At a given instant of time, t , the energy and the data traffic forecast produced by the *QoS Predictor* along with the current pattern in use, is used to determine the state, action inside the *Q-Table*. Q-Learning then selects an action from A . The corresponding pattern is then sent to the *Adaptation Manager*.

The complete algorithm of *Decision Maker* is presented in algorithm 1. It uses two key parameters, α and γ where $0 < \alpha \leq 1$ denotes the learning rate which represents the importance given to the learned observation at each step, and $0 < \gamma \leq 1$ denotes the discount factor, which can be considered as the weight given to the next action.

One of the key parts of Q-Learning is the effective use of rewards. By assigning negative rewards (penalties) to states that violate thresholds and positive rewards to states that are within thresholds, Q-Learning ensures that any incorrect decision is penalized in the form of a high negative reward and this means as the time progresses, given a state, the algorithm will be able to select the optimal adaptation pattern thereby ignoring the patterns that can lead to high penalties.

Adaptation Manager: This forms the final component of our approach, represented in Figure 2 (4). It represents the *Execute* part of the MAPE-K. It is responsible for executing the adaptation and thereby adapting the architecture. The adaptation decision produced by the *MLE* contains the pattern, P that has to be applied in the form of a configuration file. The adaptation manager then applies the adaptation by dynamically updating the existing architecture based on P .

Algorithm 1 Decision Maker Algorithm

Require: :

- 1: States $S = \{s_1, s_2, s_3, \dots, s_n\}$
 - 2: Actions $A = \{0, 1, 2, \dots, m\}$ $\triangleright$ represents the switch to each m patterns
 - 3: Thresholds $T = \{he, le, hd, ld, \dots\}$ $\triangleright$ Thresholds as per QoS requirements
 - 4: Forecasts $F = \{ef, df\}$ $\triangleright$ energy and data traffic forecasts
 - 5: Rewards $R = \{r_1, r_2, \dots, r_n\}$ $\triangleright$ Reward values
 - 6: Pattern map $P_{map} = \{p1 : 1, p2 : 2, p3 : 3, \dots, pm : m\}$ $\triangleright$ two way mapping between patterns and actions
 - 7: **procedure** DECISION-MAKER($S, A, T, F, R, p, P, \alpha, \gamma$) $\triangleright$ Find the state of the system from the forecasts
 - 8: $C \leftarrow identify_category(F, T)$
 - 9: $s \leftarrow (C, p)$ $\triangleright$ combine category and p to form s
 - 10: $r \leftarrow R[s]$ $\triangleright$ Reward for attaining the state, s
 - 11: $a \leftarrow P_{map}[p]$
 - 12: $(s', a') \leftarrow argmax_a Q(s, a)$
 - 13: $Q'(s, a) = (1 - \alpha) * Q(s, a) + \alpha * (r + \gamma * max(Q(s', a)))$
 - 14: $a \leftarrow a'$ $\triangleright$ The action to reach that state
 - 15: $p \leftarrow P_{map}[a]$
 - 16: **return** p
-

IV. IMPLEMENTATION OVERVIEW

We consider the case study mentioned in Section II for the implementation of the approach. Traditional layered architecture with enterprise-grade big data stack is used for the implementation. The development of the NdR case study and corresponding simulation is performed using CupCarbon [21]. It is considered a very good IoT simulator especially for energy simulation of IoT systems [22]. The source code (Java) of CupCarbon is modified to support dynamic pattern switch in run-time. We use *json* file for defining the energy and data traffic constraints (Thresholds, T), Elasticsearch and Kibana for data storage and visualization, Apache Kafka and Apache Spark for processing the real-time energy and data traffic logs, Python with Keras and basic Python for developing ML models and Decision Maker algorithm (Q-Learning) respectively. Further details can be found in [25].

V. EXPERIMENTATION AND RESULTS

We evaluate the approach based on its effectiveness and efficiency by answering the following research questions:

RQ1. How accurate and stable are the forecasts made by LSTM Networks?

RQ2. How much energy and data traffic is saved by the approach as compared to each individual pattern?

RQ3. How much does the adaptation quality improve with time?

RQ4. How many adaptations are performed by the approach?

RQ5. How much time does the approach takes to perform adaptation?

Datasets	Simulation Time (sec)	REL Size (MB)	AEL Size (MB)	RDFL Size (MB)	ADFL Size (MB)
CO	2,59,200	704.7	2	702.1	0.10
SU	2,59,200	497.9	2.1	520.2	0.13
SC	2,59,200	469.9	2.1	544.2	0.14
Adap	8,64,000	1771.1	5.8	1720.3	0.34

TABLE I: Summary of Datasets (REL : Raw Energy Logs, AEL : Aggregated Energy Logs, RDFL : Raw Data Traffic Logs, ADFL : Aggregated Data traffic Logs)

A. Experimental Setup

We implemented the approach on a high performance computing cluster consisting of 4 compute nodes. Each of these nodes runs on a Dell R730 CPU with an Intel Xeon Processor comprising of 20 cores with CPU 256 Gb of RAM. One node each was used for running Apache Spark, Apache Kafka, Elastic search + Kibana, and for building and testing ML models. This separation was done to mimic the real IoT data pipeline setup. We deployed the CupCarbon IoT simulator on a desktop machine running on Intel i5, 2.6-3.2 GHz processor with 16 Gb of RAM. We set the parameters $\alpha = 0.02$, $\gamma = 0.2$ and thresholds ($T = \{he = 10.0, le = 6.0, hd = 3000, ld = 2500\}$), where energy is expressed in joules and traffic in number of messages. These thresholds are defined by considering the operational constraints of the case study.

B. Data Setup

We simulate the architecture using CupCarbon simulator for 10 days. The sensor data were generated based on a Poisson distribution with mean values selected based on real observations of the case study. During the simulation, we randomly vary the patterns every 10 minutes. This is done to capture the different possible variations that might happen in the data in a real scenario. To test the stability of the forecasts made by the approach, we created three validation datasets (table I) by simulating the architecture using each of the 3 patterns for 3 days. These datasets help us to better evaluate the stability of the forecast made by our approach using the LSTM models. The complete implementation along with the source code and datasets can be found in [25].

C. Evaluation Metrics

To measure the effectiveness of the approach, we introduce four evaluation metrics i) Normalized Root Mean Square Error (NRMSE) for measuring the forecast accuracy ii) Max and Min DE which captures the exceedances of the maximum and minimum data traffic thresholds; iii) Max EE capturing the exceedances of the maximum energy thresholds.

Normalized RMSE, NRMSE for a dataset, Y with n samples is given by the formula :

$$NRMSE = \frac{RMSE(Y)}{\sigma(Y)}; RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2}$$

Where, $\sigma(y)$ represents the variance, p_i the predicted value and y_i represent the actual value.

D. Results

RQ1. How accurate and stable are the forecasts made by the LSTM Networks?

We use a forecast horizon, H , and lag value, l of 10 minutes which means that the models will forecast the energy/data traffic for the next 10 minutes based on the last 10-minute values. These are selected by considering the overfitting issues and accuracy. We use the Mean Absolute Error (MAE) loss function and the efficient Adam version of stochastic gradient descent [23] for optimization of the LSTM networks.

1. *Accuracy of energy forecasts:* We build the energy forecast model using one hidden layer consisting of 110 neurons and 220 neurons (22 components * 10 minute forecast/lag for each) in the input and output layers. These numbers were selected through experimentation and based on principles listed in [24]. The model is fit in 150 iterations. We then computed

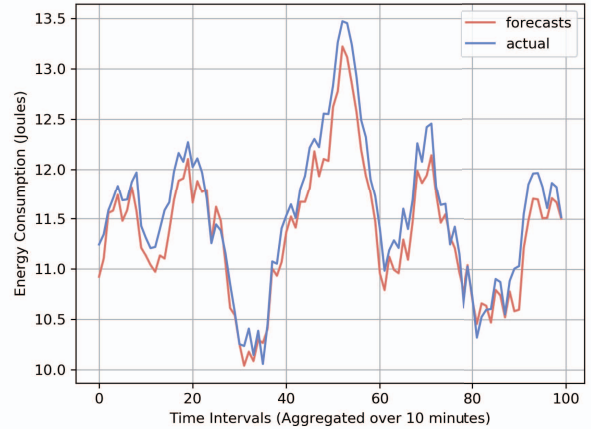


Fig. 4: LSTM Energy Forecasts

the NRMSE between the actual values and forecasted values of the testing set consisting of 4,320 samples. It resulted in a value of 0.846, Figure 4 shows the plot of actual versus forecasted values. We can see that the forecasts made by the LSTM model is almost following the curve of actual values.

2. *Accuracy of data traffic Forecasts:* For creating the data traffic LSTM model, we follow the same process as explained above by using the data traffic dataset. Since it is a univariate data, 10 neurons are used in both input and output layers. The model was fit in 100 iterations. The NRMSE calculation resulted in a value of 0.90. Figure 5 shows the plot of actual versus forecasted values. In this case, also, we can observe that all forecasts nearly follow the actual values.

3. *Stability of forecasts:* Each of these trained models is then applied to the validation sets to test the stability of the forecasts made by the respective models. The results are represented

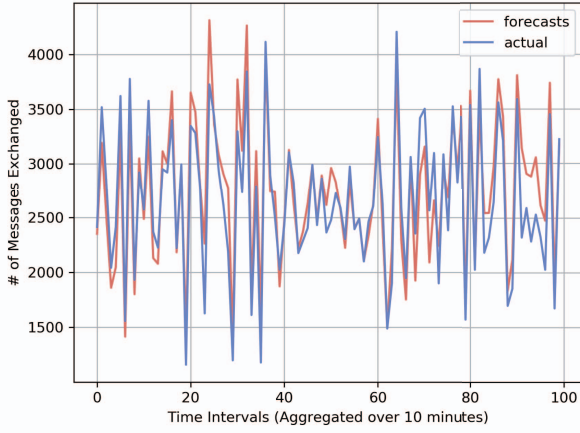


Fig. 5: LSTM Data Traffic Forecasts

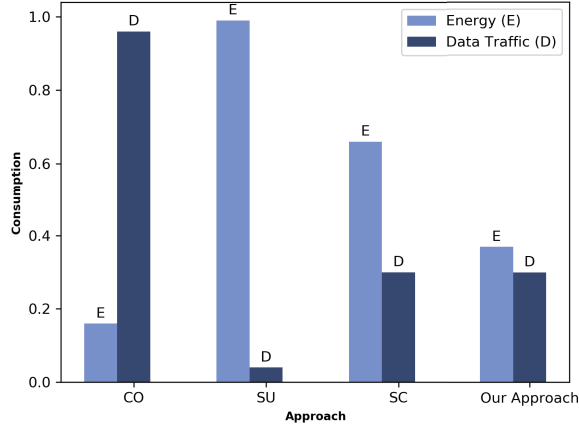


Fig. 6: Bar plot of the normalized energy and data traffic consumption of different approaches

in table II. The model is trained using Adap dataset and we can see that the accuracy level of forecasts of both energy as well as data traffic is mostly consistent across the datasets. The consistent stability is very important as the adaptation is

Dataset	NRMSE Energy	NRMSE Data
Adap	0.87	0.90
CO	0.88	0.75
SU	0.66	1.29
SC	0.50	0.76

TABLE II: Forecast stability on the validation datasets

performed based on the forecasts and instability can trigger erroneous adaptations.

RQ2. How much energy and data traffic is saved by the approach as compared to each individual pattern ?

In order to calculate the energy and data efficiency of our approach, we simulated the system using each of the three

Approach	Energy (Joules)	Traffic (# messages)	Max DE	Min DE	Max EE
CO	752.43	437885	90	0	0
SU	1995.85	382896	5	20	143
SC	1498.57	398393	14	8	104
Our Approach	1051.38	398294	18	7	8

TABLE III: Adaptation effectiveness in terms of energy and data traffic

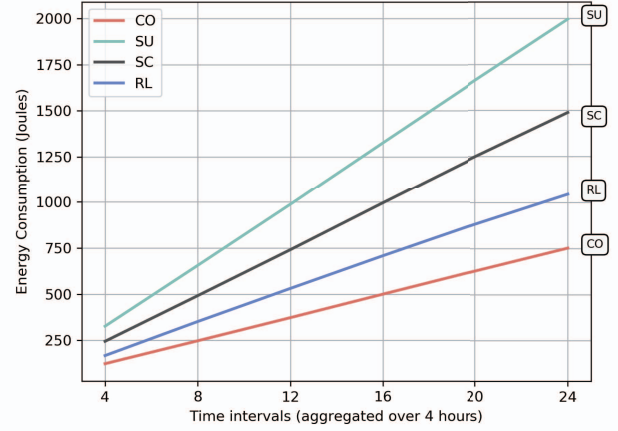


Fig. 7: Cumulative plot of energy consumption

patterns as well as with our approach and compared the results. The simulation was carried out for 1 day so as to emulate the real scenario of NdR. Table III shows the total energy consumed by the sensors and total data traffic of the system while using each of the approaches. It also shows the number of times each of the approaches exceeded the maximum and minimum thresholds. Although CO consumes the least energy and does not exceed the maximum energy limit, it results in maximum data traffic and maximum violations of Max DE. SU and SC amount to lesser data traffic but each of them goes below the Min DE. They also end up consuming more energy with SU consuming more than SC which is primarily because in SU the controller is within the sensor component. This also means they exceed the Max EE limit. Our approach, on the other hand, consumes less energy than SU and SC. It also exceeds the energy limit just 18 times. Although it consumes more energy than CO, we can see that the data traffic and Max DE is less than that of CO. Moreover, the normalized values of energy consumption and data traffic while using each of the approaches is shown in figure 6. We can see that our approach is able to guarantee better energy efficiency without compromising on data traffic (Table III). In fact, the ratio of energy saved to data traffic is almost 1 (0.66/0.7).

RQ3. How much does the adaptation quality improve with time?

In order to evaluate how the approach improves as time progresses, we computed the total energy consumption of the

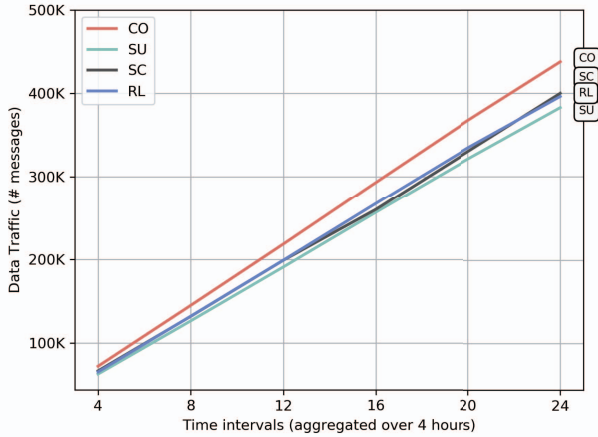


Fig. 8: Cumulative plot of data traffic

sensors and total data traffic of the system while using the approach at the intervals of 4 hours to compute how the energy and data traffic has improved throughout the day. This was then compared with each of the individual patterns. The results are reported in figures 7 and 8 in the form of cumulative energy and data traffic plots. For the first interval, the energy consumed using our approach (ER) is 167.62 Joules and total data traffic (DR) is 65684. This value of data traffic is just 6789 less than that of CO (The pattern with highest data traffic) and energy is 159 Joules less than SU (the pattern with least data traffic). This margin keeps improving and as time progresses we can see that the value of data traffic becomes much lesser than CO and the energy consumption becomes lesser from SU. This holds true with SC as well. The reason for this behavior can be attributed to the ability of the decision-maker to keep improving based on the feedback through rewards. The approach thus ensures that the system keeps learning and improving with time by reducing energy consumption while keeping the data traffic within the limits.

RQ4. How many adaptations are performed by the approach?

We evaluate the adaptation efficiency in terms of the total number of adaptations performed by the approach. Towards this, we computed the number of adaptations performed during each of the 4-hour intervals. For each interval, the average number of adaptation performed amounts to 13 with the total number of adaptations amounting to 79. On the other hand, every interval amounts to 240 minutes (240 possible adaptations) and this implies that the approach is able to intelligently perform the adaptation only when required based on the forecast. This can be clearly attributed to the ability of LSTM's to provide accurate forecasts and that of the decision-maker to proactively generate the best adaptation decisions.

RQ5. How much time does the approach takes to perform adaptation?

To measure adaptation efficiency in terms of the time consumed per adaptation, we clocked the average time taken

by the approach to generate the forecast, select a decision, and execute the adaptation. The results show that on average the approach takes 0.23 seconds for performing the whole process. The speed can be primarily attributed to an efficient data pipeline which makes use of some enterprise-grade big data stack. The fact that Q-Learning being a model-free technique also contribute to the speed of the approach. The majority of time is consumed by the adaptation executor which averages to around 0.13 seconds as it has to transfer the adaptation decision over the network. So, on a whole, we can observe that the approach is able to perform adaptations in near real-time for the given study.

E. Threats to Validity

The threats to *internal validity* are related to the incorrect forecast/incorrect decisions. In order to improve the accuracy, the build phase is executed (as mentioned in Section III.C and IV) in parallel at regular intervals. The incorrect decision issue is handled implicitly through the proactive nature of our approach. Further, by keeping the reward for incorrect decisions to be a high negative value, wrong decisions can be heavily penalized.

Threats to *external validity* concerns the generalizability and scalability of our approach. Although our approach has been applied to a specific case study, the techniques used by our approach can be replicated in any case study/use-case as long as we have a way to generate the QoS data. Our approach uses generic machine-learning techniques. Further, the threshold limits and reward values can be modified depending on the use-case. In terms of scalability, we believe that our approach can be applied to more complex systems with optimization of Q-Learning. Also, the layered architecture used for implementation supports both horizontal and vertical scalability.

VI. RELATED WORK

There has been some works on self-adaptive systems for energy reduction in IoT systems. Massimiliano Raciti in his thesis proposed an approach based on anomaly detection to detect security vulnerabilities of a cyber-physical system (CPS) [31]. A generic context aware adaptive framework for reducing energy consumption in Machine to Machine communication systems was proposed by cost et al. [32]. Moreno-Cano et al. came up with a smart environment management architecture supported with IoT capabilities to achieve smart and sustainable environments [33]. Wunderlich et al [34] propose a way to improve the performance in communication among IoT devices and thereby improve energy efficiency using network coding techniques. Seiger et al. propose a self-adaptive workflows for CPS using the MAPE-K loop [35].

Some papers on the use of ML for self-adaptive systems in other domains have been presented as well. RL-based approaches for planning and adaptation in software systems were presented by Dongsun Kim et al. [26] and Han Nguyen Ho et al. [27]. An Adaptation Approach for Self-Adaptive Sensor Networks using predictions was proposed by Anaya et al. in [28]. An analysis of different decision-making techniques

in dynamic self-adaptive systems was performed by Pawel Idziak et. al [29]. Tao Chen et al. [30] presented a multi-learner approach for self-adaptive and online QoS Modeling for cloud-based software services.

Differently from the related works reported above, our approach identifies the need for adaptation at an early stage using ML models and uses RL techniques (as compared to the traditional techniques) to determine the optimal adaptation strategy. The feedback of the strategy selected is used to improve the decision-making process, thereby making it smarter.

VII. CONCLUSIONS AND FUTURE WORK

We demonstrated that our approach, by using ML techniques can improve the energy efficiency of an IoT system without compromising on data traffic. It achieves this by providing accurate energy and data traffic forecasts with a normalized error of 0.87 and 0.90 respectively. We also demonstrated that the approach can perform effective as well as efficient adaptations. We believe that is a starting point and there is more to be done in the future. These include, but are not limited, to:

Considering more QoS parameters and patterns. One of our immediate future goals is to extend and validate this approach by identifying more patterns and for different QoS parameters like response time, utilization, throughput, etc.

Handling of unknown's. Extend this approach to support reactive adaptation capabilities to ensure stability in the event of a sudden unforeseen situation like sensor failures.

REFERENCES

- [1] Stack, T., 2020. Iot Data Continues To Explode Exponentially. Who Is Using That Data And How? - Cisco Blogs. [online] Cisco Blogs. Available at: <https://tinyurl.com/y6bf2cgj> [Accessed 12 February 2020].
- [2] Newsroom.cisco.com. 2020. Cisco Survey Reveals Close To Three-Fourths Of Iot Projects Are Failing. [online] Available at: <https://tinyurl.com/yaxqsdxf> [Accessed 12 February 2020].
- [3] Ciccuzzi, Federico, Ivica Crnkovic, Davide Di Ruscio, Ivano Malavolta, Patrizio Pelliccione, and Romina Spalazzese. "Model-driven engineering for mission-critical iot systems." IEEE software 34, no. 1 (2017): 46-53.
- [4] Hammoudi, Sarra, Zibouda Aliouat, and Saad Harous. "Challenges and research directions for Internet of Things." Telecommunication Systems 67, no. 2 (2018): 367-385.
- [5] Hassan, Zozo & Ali, Hesham & M Badawy, Mahmoud. (2015). Internet of Things (IoT): Definitions, Challenges, and Recent Research Directions. International Journal of Computer Applications. 128. 975-8887.
- [6] Mazeiar Salehie and Ladan Tahvildari. Self-adaptive software: Landscape and research challenges. ACM Trans. Auton. Adapt. Syst., 4(2):14:1-14:42, May 2009. ISSN 1556-4665.
- [7] Weyns, Danny. "Software engineering of self-adaptive systems: an organised tour and future challenges." Chapter in Handbook of Software Engineering (2017).
- [8] Krupitzer, Christian, Felix Maximilian Roth, Sebastian VanSyckel, Gregor Schiele, and Christian Becker. "A survey on engineering approaches for self-adaptive systems." Pervasive and Mobile Computing 17 (2015): 184-206.
- [9] Atzori, Luigi, Antonio Iera, and Giacomo Morabito. "The internet of things: A survey." Computer networks 54, no. 15 (2010): 2787-2805.
- [10] Gubbi, Jayavardhana, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. "Internet of Things (IoT): A vision, architectural elements, and future directions." Future generation computer systems 29, no. 7 (2013): 1645-1660.
- [11] Musil, Angelika, Juergen Musil, Danny Weyns, Tomas Bures, Henry Muccini, and Mohammad Sharaf. "Patterns for self-adaptation in cyber-physical systems." In Multi-disciplinary engineering for cyber-physical production systems, pp. 331-368. Springer, Cham, 2017.
- [12] Andrae, Anders & Edler, Tomas. (2015). On Global Electricity Usage of Communication Technology: Trends to 2030. Challenges. 6. 117-157.
- [13] Malmodin, Jens, and Dag Lundén. "The energy and carbon footprint of the global ICT and E&M sectors 2010-2015." Sustainability 10, no. 9 (2018): 3027.
- [14] Kephart, J. O., Chess, D. M. (2003). The vision of autonomic computing. Computer, (1), 41-50.
- [15] Henry Muccini and Karthik Vaidhyathan, "PIE-ML: A Machine learning-driven Proactive Approach for Architecting Self-adaptive Energy Efficient IoT Systems", DISIM, University of L'Aquila, L'Aquila, Italy, TRCS: 002/2020, July. 15, 2020.[Online]. Available: <https://tinyurl.com/y98weaat>
- [16] Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." Neural computation 9, no. 8 (1997): 1735-1780.
- [17] Hochreiter, S., Bengio, Y., Frasconi, P. and Schmidhuber, J., 2001. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies.
- [18] Xingjian, S. H. I., Zhouong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. "Convolutional LSTM network: A machine learning approach for precipitation nowcasting." In Advances in neural information processing systems, pp. 802-810. 2015.
- [19] Sutton, Richard S., and Andrew G. Barto. Introduction to reinforcement learning. Vol. 135. Cambridge: MIT press, 1998.
- [20] Tokic, Michel, and Haitham Bou Ammar. "Teaching reinforcement learning using a physical robot." In Proceedings of the ICML Workshop on Teaching Machine Learning. 2012.
- [21] Cupcarbon - A Smart City Iot WSN Simulator. [online] Available at: <http://www.cupcarbon.com> [Accessed 12 Feb 2020].
- [22] Lopez-Pavon, Cristina & Sendra, Sandra & F. Valenzuela-Valdes, Juan. (2018). Evaluation of CupCarbon Network Simulator for Wireless Sensor Networks. Network Protocols and Algorithms.
- [23] Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." arXiv preprint arXiv:1412.6980 (2014).
- [24] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. Deep learning. MIT press, 2016.
- [25] Implementation - https://github.com/karthikv1392/IoTAdaptaiton_ML
- [26] Kim, Dongsun, and Sooyong Park. "Reinforcement learning-based dynamic adaptation planning method for architecture-based self-managed software." In Software Engineering for Adaptive and Self-Managing Systems, 2009. SEAMS'09. ICSE Workshop on, pp. 76-85. IEEE, 2009.
- [27] Han Nguyen Ho and Eunseok Lee. Model-based reinforcement learning approach for planning in self-adaptive software system. In Proceedings of the 9th International Conference on Ubiquitous Information Management and Communication, page 103. ACM, 2015.
- [28] Anaya, Ivan Dario Paez, Viliam Simko, Johann Bourcier, Noël Plouzeau, and Jean-Marc Jézéquel. "A prediction-driven adaptation approach for self-adaptive sensor networks." In Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 145-154. ACM, 2014.
- [29] Pawel Idziak and Siobhan Clarke. An analysis of decision-making techniques in dynamic, self-adaptive systems. In Self-Adaptive and Self-Organizing Systems Work-shops (SASOW), 2014 IEEE Eighth International Conference on, pages 137-143. IEEE, 2014.
- [30] Tao Chen and Rami Bahsoon. Self-adaptive and online qos modeling for cloud-based software services. IEEE Transactions on Software Engineering, 43(5):453-475, 2017.
- [31] Raciti, Massimiliano. "Anomaly detection and its adaptation: Studies on cyber-physical systems." PhD diss., Linköping University Electronic Press, 2013.
- [32] J. M. Costa and G. Miao, "Context-aware Machine-to-Machine communications," 2014 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Toronto, ON, 2014, pp. 730-735.
- [33] Moreno-Cano, María V., Jose Santa, Miguel A. Zamora-Izquierdo, and Antonio F. Skarmeta. "Future human-centric smart environments." In Modeling and Processing for Next-Generation Big-Data Technologies, pp. 341-365. Springer, Cham, 2015.
- [34] Wunderlich, Simon, Juan A. Cabrera, Frank HP Fitzek, and Martin Reisslein. "Network coding in heterogeneous multicore IoT nodes with DAG scheduling of parallel matrix block operations." IEEE Internet of Things Journal 4, no. 4 (2017): 917-933.
- [35] Seiger, Ronny, Steffen Huber, Peter Heisig, and Uwe Aßmann. "Toward a framework for self-adaptive workflows in cyber-physical systems." Software & Systems Modeling 18, no. 2 (2019): 1117-1134.